Proyecto individual
Pipeline de Monitorización con Kafka y MongoDB

ÍNDICE

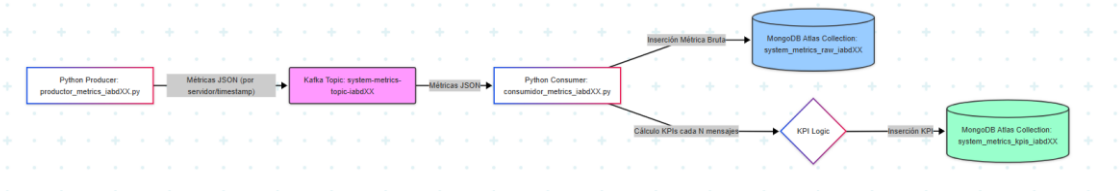
Descripción 3
Objetivo principal 3
Repositorio base 3
Modalidad 4
Requisitos técnicos 4
Qué debes implementar 4
Tareas a realizar 5
Entregables 6
Criterios de evaluación 6
Evidencias recomendadas 7
Notas 7

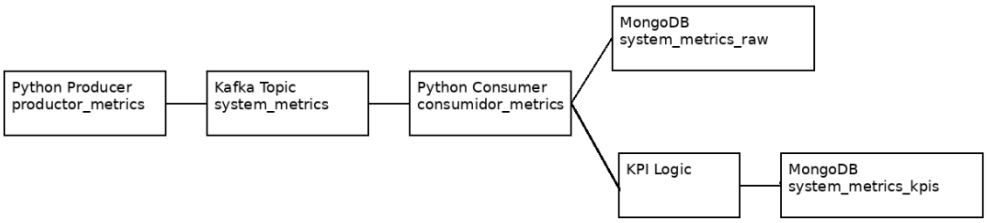
Descripción

En esta tarea realizarás un proyecto práctico individual sobre **procesamiento de datos en tiempo real en sistemas Big Data**, trabajando con un ecosistema basado en **Apache Kafka, Python y MongoDB Atlas**.

Vamos a simular la monitorización de una infraestructura simple. Varios servidores (simulados por un productor) reportan métricas de rendimiento a un sistema centralizado a través de Kafka.

Un consumidor recogerá estas métricas, las almacenará en bruto y calculará KPIs agregados para analizar el estado del sistema.





Objetivo principal

Implementar un sistema completo que:

- Genere métricas simuladas de varios servidores
- Las envíe a Kafka
- Las consuma desde Python
- Las almacene en MongoDB Atlas
- Calcule KPIs agregados
- Almacene los KPIs en una segunda colección

Repositorio base

Disponible en GitHub (fork obligatorio). https://github.com/josedavidmi/kafka-monitoring-lab.git

Incluye:

- docker-compose (Kafka)
- scripts base
- documentación
- estructura del proyecto

Modalidad

Trabajo individual mediante:

- Fork del repositorio
- Desarrollo sobre tu repositorio
- Entrega mediante GitHub

Requisitos técnicos

Kafka

```
docker compose up -d
```

Broker:

```
iplocal:29092
```

MongoDB Atlas

Uso obligatorio de conexión remota mediante:

```
CONNECTION_STRING
```

Qué debes implementar

<u>1º) Productor</u>

Simular servidores:
web01, web02, db01, app01, cache01

Generar métricas:

- cpu_percent
- memory_percent
- disk_io_mbps
- network_mbps
- error_count

Formato del mensaje:

```
{
  "server_id": "...",
  "timestamp_utc": "...",
  "metrics": { ... },
  "message_uuid": "..."
}
```

Enviar a Kafka:

```
system-metrics-topic
```

2º) Consumidor

- Leer mensajes desde Kafka
- Usar group_id único
- Insertar en:
  `system_metrics_raw`

3º) KPIs (ventana tumbling)

Cada **20 mensajes**:
Calcular:

- promedio CPU
- promedio memoria
- promedio disco
- promedio red
- suma de errores
- tasa de procesamiento:
  `mensajes / segundos`

Guardar en:

```
system_metrics_kpis
```

Cada documento debe incluir:

- timestamp
- duración ventana
- número de mensajes
- KPIs

Tareas a realizar

Parte A: Productor

- Generación de métricas
- Serialización JSON
- Envío a Kafka

- Manejo de errores

Parte B: Consumidor

- Consumo desde Kafka
- Inserción en MongoDB
- Cálculo de KPIs
- Manejo de errores
- Control de offsets

Entregables

1. Repositorio GitHub (fork)

2. Tag obligatorio:
```
git tag v1.0-entrega
git push origin v1.0-entrega
```

3. Evidencias en repo:
   - README.md
   - docs/evidencias.md
   - Scripts funcionales
   - Datos en MongoDB

Criterios de evaluación

A. Productor (Kafka) — 2.5 pts
- Genera mensajes correctos
- Serializa JSON correctamente
- Intervalo y servidores correctos
- Manejo de errores

B. Consumidor (Kafka → MongoDB) — 2.5 pts
- Usa group_id
- Inserta datos sin pérdida
- Maneja offsets correctamente
- Reconexión básica

C. KPIs — 2.0 pts
- Ventana tumbling de 20 mensajes
- Cálculo correcto de métricas
- Inserción correcta en MongoDB

D. Calidad del código — 1.0 pts
- Código modular
- Variables claras

- Uso de .env
- README correcto

E. Logging — 0.5 pts

- Logs claros
- Información útil

F. Entregables — 0.5 pts

- Repo completo
- Evidencias claras

G. Bonus — +1.0 pts

- Dockerización
- Manejo avanzado de errores
- Tests
- Documentación extra

Evidencias recomendadas

- Productor enviando datos
- Consumidor funcionando
- MongoDB RAW
- MongoDB KPIs
- Logs

Notas

- Usar UTC: `datetime.now(timezone.utc)`
- Ventana = tumbling
- Se valora código reproducible
- Se valora claridad